



Licenciatura em Engenharia Informática
Projeto de Engenharia Informática em Contexto Empresarial
3º ano

Fábio Miguel Domingues Monteiro

DESENVOLVIMENTO DE UM CHATBOT INTELIGENTE

**Relatório de projeto orientado pelo Professor Doutor Justino Lourenço e
apresentado ao Instituto Superior Politécnico Gaya**

07 de 2026

Dedicatória

Dedico este relatório aos meus pais, irmãos e namorada pelo apoio incondicional que sempre me deram durante o meu percurso académico, contribuindo também para o meu crescimento pessoal.

Agradecimentos

Começo por agradecer ao Prof. Dr. Sérgio Sargo e ao Prof. Dr. Justino Lourenço, coordenadores do Instituto Politécnico Gaya pela orientação dada durante o meu percurso académico.

Um agradecimento especial à WideSkills, Lda. pelo voto de confiança que me deram para desenvolver este projeto, no qual adquiri diversas competências profissionais.

Uma enorme gratidão à minha família pelo amor incondicional e motivação que me deram durante esta etapa importante da minha vida.

Resumo

Este relatório descreve toda a ideia e desenvolvimento de uma solução de apoio à secretaria, feita especificamente para a secretaria do Instituto Superior Politécnico Gaya (ISPGAYA). Os serviços académicos recebem um considerado volume de emails, grande parte deles referentes a dúvidas repetitivas sobre normas, preços e prazos e a necessidade de consultar constantemente diversos regulamentos para formular respostas corretas consome muito tempo aos funcionários do instituto.

Para resolver este problema, o projeto foca-se em implementar um assistente virtual que, através de dados exportados pelo site do ISPGAYA, dá soluções de respostas a emails enviados para o instituto. Portanto, o sistema recolhe e lê a documentação e texto presente no site do instituto e cria uma base de conhecimento. Sempre que um estudante envia uma dúvida, a ferramenta analisa o conteúdo da mensagem, compreende a intenção e procura soluções de resposta exata através dos regulamentos da escola.

Em vez de comunicar diretamente com o utilizador final, o sistema cria apenas um email na caixa de entrada do instituto a mencionar o pedido do aluno e soluções de resposta ao mesmo. De seguida, cabe à equipa da secretaria ler, validar e aprovar o envio. Esta metodologia elimina o risco de partilha de informações incorretas e garante que o atendimento se mantém fidedigno.

A implementação deste modelo é uma prova de modernização dos processos internos, pois reduz drasticamente o tempo utilizado na pesquisa de documentos, a solução diminui a carga dos colaboradores, permite respostas muito mais rápidas aos estudantes e liberta os colaboradores para um atendimento mais descansado e focado em problemas complexos.

Abstract

This report describes the concept and development of an administrative support solution designed specifically for the administrative office of the Instituto Superior Politécnico Gaya (ISPGAYA). The academic departments receive a significant volume of email inquiries, most of which concern repetitive questions about policies, prices, and deadlines. The need to constantly consult various regulations to formulate accurate responses is very time-consuming for the institute's staff.

To address this issue, the project focuses on implementing a virtual assistant that, using data exported from the ISPGAYA website, provides responses to emails sent to the institute. Thus, without manual intervention, the system collects and interprets the institution's official documentation and public text to build an up-to-date knowledge base. Whenever a student submits a question, the tool analyzes the message's content, understands the intent, and searches for precise answers based on the school's regulations.

Instead of communicating directly with the end user, the system simply creates an email in the institute's inbox mentioning the student's request and possible responses to it. It is then up to the administrative staff to read, validate, and approve the email for sending. This approach eliminates the risk of sharing incorrect information and ensures that the service remains reliable.

The implementation of this model demonstrates the modernization of internal processes, as it drastically reduces the time spent searching for documents; the solution lightens the workload on staff, enables much faster responses to students, and frees up staff to provide more relaxed service focused on complex issues.

Lista de abreviaturas e siglas

ISPGAYA Instituto Superior Politécnico Gaya.

RAG Retrieval-Augmented Generation.

PLN Processamento de Linguagem Natural.

LLM Large Language Model.

IA Inteligência Artificial.

API Application Programming Interface.

Índice

Dedicatória	iii
Agradecimentos	iv
Resumo	v
Abstract	vi
Lista de abreviaturas e siglas.....	vii
Introdução.....	11
Caracterização da entidade	12
Bases teóricas	12
Requisitos Funcionais.....	13
Requisitos Não-Funcionais	13
Análise e descrição da arquitetura do sistema	14
Descrição do trabalho/atividades realizadas	15
1.ª Fase: Web Scraping.....	15
2.ª Fase: Configuração do Microsoft Azure	16
3.ª Fase: Desenvolvimento do sistema de resposta a emails	16
4.ª Fase: Interface Auxiliar de Verificação	16
Resultados obtidos.....	17
Testes de validação	17
Extração de informações.....	17
Organizar dados.....	19
Identificação da intenção do remetente	20
Pesquisa	22
Gerar resposta	23
Cronograma.....	25
Problemas e decisões.....	26
Limitações na extração de dados com o <i>Power Automate</i>	26
Restrições no Microsoft Copilot.....	26
Limitações de espaço de ficheiros importados no Azure	26
Análise de resultados.....	27
1. Criação e Estruturação da Base de Conhecimento	27
2. Gerar respostas	27
3. Impacto na Eficácia da Comunicação.....	27
Demonstração visual do projeto.....	28

Conclusões	30
Bibliografia	31
Glossário	31
Apêndices/anexos	31

Índice de apêndice/anexo

Ilustração 1	31
--------------------	----

Índice de figuras

Figura 1	12
Figura 2	14
Figura 3	15
Figura 4	28
Figura 5	29
Figura 6	29

Índice de tabelas

Tabela 1	17
Tabela 2	18
Tabela 3	18
Tabela 4	19
Tabela 5	19
Tabela 6	20
Tabela 7	20
Tabela 8	21
Tabela 9	21
Tabela 10	22
Tabela 11	22
Tabela 12	23
Tabela 13	23
Tabela 14	24
Tabela 15	24
Tabela 16	25

Introdução

Este trabalho, pretende dar conta de um projeto prático que se enquadra no relatório final da unidade curricular de Projeto de Engenharia Informática em Contexto Empresarial, do 3º ano da Licenciatura em Engenharia Informática. Este projeto, centrado no campo da informática e da inteligência artificial, procura contribuir para uma compreensão mais aprofundada sobre os processos de automatização no atendimento administrativo via email. O problema central é o elevado número de emails recebidos pela secretaria do *ISPGAYA*. As dúvidas dos alunos podem ser repetitivas e há certos temas que são muito abordados, como: preços, bolsas ou regras gerais. Como resposta a esta necessidade, o projeto desenvolvido é uma ferramenta tecnológica que consegue ler esses emails, procurar respostas através da informação exportada do site oficial do instituto e preparar soluções de respostas de forma automática.

O desenvolvimento deste projeto contou com o apoio e o voto de confiança da WideSkills, Lda., que me permitiu adquirir diversas competências profissionais ao longo deste percurso.

Desde o início deste projeto, pretendeu-se:

- Implementar um mecanismo automático de extração de informações e documentos a partir do site do *ISPGAYA*.
- Criar uma base de conhecimento na nuvem, através do *Azure AI Search*, para centralizar a informação.
- Desenvolver uma *API* capaz de gerir a comunicação entre os serviços.
- Gerar soluções de resposta precisos e baseados nas informações retidas, com a utilização de um modelo da *OpenAI*.
- Reduzir a carga de trabalho dos funcionários, mesmo que tenham a função de garantir e enviar o email final.

Caracterização da entidade

O projeto interno foi desenvolvido em colaboração com a *WideSkills Lda*, cuja sede é no *ISPGAYA*.

Dentro da sua vertente tecnológica, destaca-se pela conceção e desenvolvimento de projetos de eletrónica e automatização.

Figura 1

Sede WideSkills Lda. ISPGAYA



Bases teóricas

Para realizar a componente prática deste projeto foi essencial pesquisar mais acerca de conceitos tecnológicos necessários e solicitados no plano do projeto.

O primeiro conceito importante é a automatização de processos, que permite captar eventos em tempo real, processar e responder.

O segundo conceito teórico é o Processamento de Linguagem Natural (PLN). Com o avanço dos *Large Language Model* (LLM), agora é possível uma IA ler uma mensagem de um aluno e perceber o contexto real, consegue até ignorar erros ortográficos, compreender abreviaturas e focar-se apenas na intenção principal do remetente.

No entanto, o uso de modelos de linguagem tem um problema, como vai buscar dados abertos fornecidos pela internet, eles podem inventar respostas quando não sabem a resposta especificamente. Para resolver isto, é utilizada uma arquitetura chamada *Retrieval-Augmented Generation* (RAG), que primeiro faz uma pesquisa numa base de dados interna para depois entregar esse texto ao modelo. Isto garante a precisão da resposta.

Por fim, o projeto também tem técnicas de *Web Scraping*, que consiste em percorrer páginas web de forma autónoma. É feita a leitura da estrutura das páginas, encontra as ligações e transfere os ficheiros e texto. Esta extração de dados substitui trabalho manual de procurar e guardar centenas de regulamentos em *PDF's*.

Requisitos Funcionais

- Extração de informações: O sistema deve extrair todo o texto e documentos automaticamente do website do instituto, tanto em português como em inglês e ignorar ligações, como o inforestudante e outros sites com necessidade de login.
- Organizar dados: Como o *Microsoft Azure*, na importação de ficheiros, tem um limite de armazenamento por cada um, foi programado para a extração ser feita em mais do que um ficheiro *.json* para não ultrapassar esse limite.
- Identificação da intenção do remetente: Através da *IA*, o sistema deve interpretar emails para extrair palavras-chave e identificar a intenção da solicitação.
- Pesquisa: O sistema deve cruzar as palavras identificadas com a base de conhecimento indexada no *Azure Search* para recuperar o contexto institucional.
- Gerar resposta: O sistema deve disponibilizar uma *API* que, ao receber um email, forneça exemplos de resposta com base apenas nos documentos e texto exportados do website do instituto, mantendo o histórico da conversa e idioma da pergunta, para os funcionários da secretaria tomarem uma decisão e responderem de forma mais rápida.

Requisitos Não-Funcionais

- Desempenho e tempo de resposta: Receber a resposta não deve exceder um tempo de espera que comprometa a fluidez da comunicação.
- Tolerância e falhas: Os *scripts* de extração de dados devem ter mecanismos de tratamento de erros para ignorar páginas inválidas sem interromper o processo global.
- Informação confiável: As respostas da *IA* devem ser estritamente dadas através dos dados institucionais. Caso não encontre uma resposta para o tema solicitado, assume falta de informação, o que faz com que haja a certeza de que todas as respostas são fiáveis.

Análise e descrição da arquitetura do sistema

A arquitetura do projeto foi desenhada com base no padrão tecnológico RAG.

1. Camada de Processamento: Corre os *scripts Python* dedicados ao *web scraping* e leitura de ficheiros.
2. Camada de Armazenamento e Indexação: Os dados são carregados para um serviço de pesquisa no *Microsoft Azure*. Este componente coloca o texto em vetores, o que permite uma pesquisa não só pelas palavras-chave, mas também pelo significado semântico.
3. Camada de Geração: Utiliza o *LLM* para interpretar as intenções e idioma das solicitações iniciais e cruzar o texto final com o pedido.
4. Camada de Apresentação: Depois de processar o email recebido, fornece respostas possíveis.

Figura 2

Diagrama de Casos de Utilização

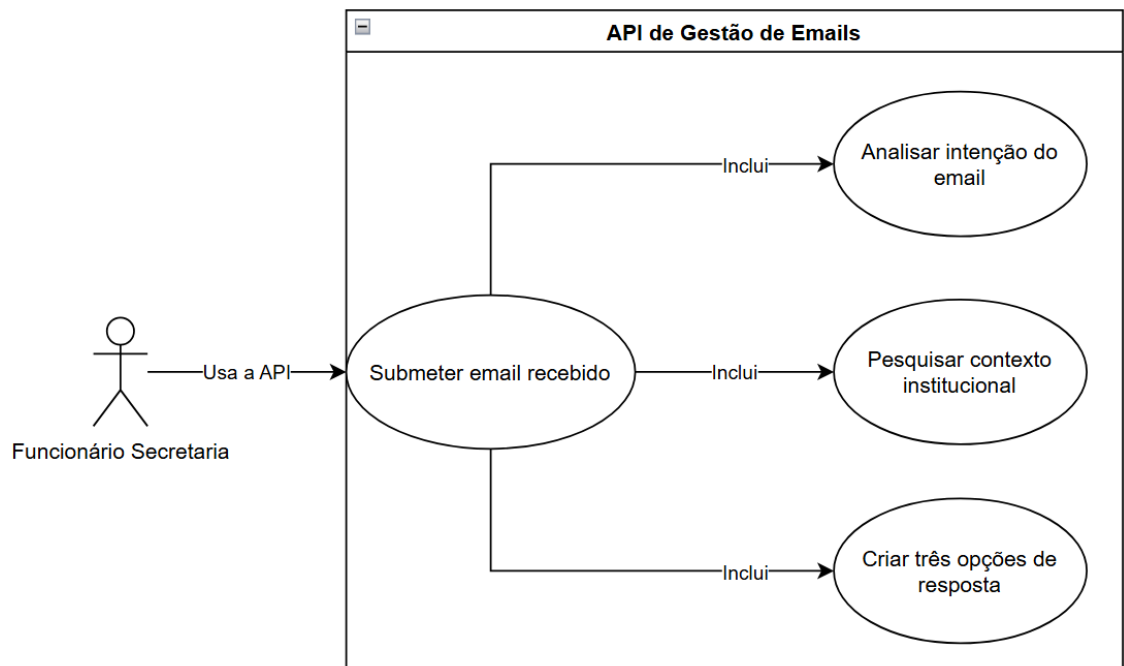
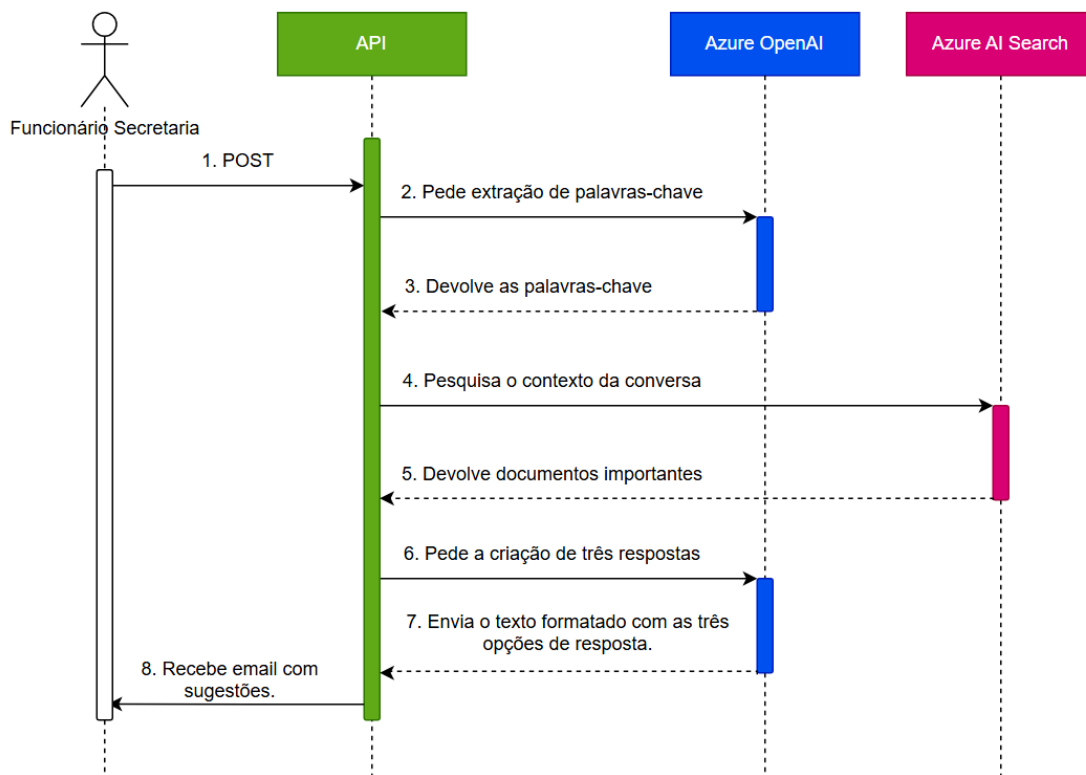


Figura 3*Diagrama de Sequência*

Descrição do trabalho/atividades realizadas

Este projeto foi desenvolvido através de várias etapas realizadas, desde a recolha de dados até ao serviço de IA para resposta a emails. A implementação foi totalmente desenvolvida em linguagem *Python*, aproveitando diversas bibliotecas especializadas aos serviços necessários e com integração com os serviços *cloud* do *Microsoft Azure*.

1.^a Fase: Web Scraping

O primeiro passo foi recolher toda a informação do website oficial do ISPGAYA, de forma a criar uma base de conhecimento necessária para a IA conseguir dar respostas contextualizadas e fiáveis.

Para isso, foram desenvolvidos três scripts em python distintos:

- Extração Estática (*extrator.py*): Foi utilizada a biblioteca *requests* para aceder às páginas *web* e a *BeautifulSoup* para fazer a análise sintática do código *HTML* das mesmas.
- Extração Dinâmica (*extrair_dinamico.py*): Uma vez que o repositório de documentos do site do ISPGAYA é gerado dinamicamente, que impede a utilização da biblioteca *BeautifulSoup*, foi utilizada a *Playwright*. Este *script* simula o uso de um *browser* real, clica automaticamente nas pastas e subpastas e descarrega todos os ficheiros pdf encontrados, não repetindo se encontrar iguais.

- Processamento de Ficheiros (*leitor_pdfs.py*): Com os ficheiros já exportados, este modelo utiliza a biblioteca *PyPDF2* para abrir cada *pdf*, extrair o texto de todas as suas páginas e associa esse conteúdo a um tema derivado do nome do ficheiro. De seguida, guarda o resultado num formato *JSON* compatível com o *Microsoft Azure*.
- Segmentação de Dados (*cortador.py*): Para garantir que os limites de importação dos ficheiros no *Azure* não são excedidos, foi desenvolvido um código que faz com que haja uma divisão de informações (ficheiros com um número limite de entradas cada).

2.ª Fase: Configuração do Microsoft Azure

Com os dados já estruturados e segmentados, procedeu-se à sua integração no *Azure*.

- Azure AI Search: Os ficheiros *JSON* foram carregados para a criação de um índice de pesquisa. Este serviço indexa o texto e permite pesquisas rápidas sobre a documentação do instituto.
- Azure OpenAI: Foi escolhido e utilizado o modelo LLM (gpt 5.4 mini) que faz o processamento do texto solicitado e o envio de soluções de resposta.

3.ª Fase: Desenvolvimento do sistema de resposta a emails

A parte central do projeto passa pela criação da API de emails (*api_email.py*), o componente criado para a gestão e automatização.

Foi desenvolvido com a *framework FastAPI*, o sistema trabalha da seguinte forma quando recebe um pedido:

- Verifica a Intenção: O texto do email recebido é enviado para o modelo da *OpenAI* com um *prompt* específico que o obriga a extrair as palavras-chave (tanto em português como em inglês).
- Recuperação de Contexto (*RAG*): As palavras-chave são enviadas para o *Azure AI Search* que devolve, de seguida, os fragmentos de texto extraídos previamente do *website* e dos *pdf's*.
- Resposta formatada: A *IA* recebe o contexto institucional cruzado com o email original e dá, exatamente, três opções de resposta.

4.ª Fase: Interface Auxiliar de Verificação

Como complemento e ferramenta de teste ao longo do desenvolvimento deste projeto, foi feita também um *script* secundário (*app_chat.py*), com a utilização da biblioteca *Streamlit*. Esta interface web foi criada para interagir, através de um chat, com a *IA* de uma forma mais rápida com o intuito de verificar e validar a coerência das respostas.

Resultados obtidos

O desenvolvimento das fases mencionadas anteriormente resultou na criação de um sistema funcional e resiliente. O processo de extração reuniu todas as páginas e *pdfs* institucionais numa base de conhecimento centralizada, a *API* desenvolvida demonstrou ser capaz de receber solicitações por email e devolver, em segundos, respostas prontas a utilizar, formatadas corretamente e fundamentadas em dados reais, com o cumprimento do objetivo de melhorar a eficácia e reduzir o tempo de resposta do atendimento via email.

Testes de validação

Neste capítulo são apresentados testes que foram realizados ao longo do desenvolvimento do projeto que validam o funcionamento correto dos requisitos necessários.

Extração de informações

Tabela 1

Caso de teste 1

ESPECIFICAÇÃO DE CASO DE TESTE 1	
Identificador:	PROJECT-EXT-001
Objetivo:	Verificar a extração do texto português e inglês do website do ISPGAYA.
Autor(es)	Fábio Monteiro
Especificação de Entradas Executar o script criado através do link principal do site do instituto.	
Especificação de Saídas a. O sistema percorre todas as páginas, menos as de uma lista criada como 'lista negra' com as páginas do inforestudante, Outlook e links de terceiros. b. Todo o texto, tanto no site em português como em inglês, é extraído e guardado em ficheiros <i>JSON</i> .	
Outros NA	
Dependências Ligação à internet e servidor web do instituto aberto.	

Tabela 2

Caso de teste 2

ESPECIFICAÇÃO DE CASO DE TESTE 2	
Identificador:	PROJECT-EXT-002
Objetivo:	Verificar se o programa dividiu bem os ficheiros e idiomas.
Autor(es)	Fábio Monteiro
Especificação de Entradas Executar o script que organiza os dados em ficheiros <i>json</i> diferentes, para português como para inglês.	
Especificação de Saídas a. O sistema divide os dados b. São criados ficheiros com as informações exportadas do site de forma organizada.	
Outros Garante compatibilidade com o <i>Azure</i> .	
Dependências NA	

Tabela 3

Caso de teste 3

ESPECIFICAÇÃO DE CASO DE TESTE 3	
Identificador:	PROJECT-EXT-003
Objetivo:	Verificar a procura dinâmica de extração de documentos no repositório de regulamentos na página web do <i>ISPGAYA</i> (tem de conseguir abrir as pastas e subpastas para transferir todos os ficheiros).
Autor(es)	Fábio Monteiro
Especificação de Entradas Executar o script de extração dinâmica com base no link de repositórios de documentos, com base no nome das pastas principais para depois correr por todas as subpastas e transferir todos os documentos necessários.	
Especificação de Saídas a. O sistema lê e acede autonomamente às pastas e ficheiros. b. Todos os documentos encontrados são transferidos para uma pasta do projeto, ignora se houver ficheiros iguais.	
Outros NA	
Dependências Playwright devidamente programado e ligação à internet.	

Organizar dados

Tabela 4

Caso de teste 1

ESPECIFICAÇÃO DE CASO DE TESTE 1	
Identificador:	PROJECT-ORG-001
Objetivo:	Verificar se o programa fez, de forma correta, a divisão do json completo de cada idioma para ficheiros menores. Isto garante que seja possível importar os ficheiros no <i>Azure</i> .
Autor(es)	Fábio Monteiro
Especificação de Entradas Executar o script que divide o ficheiro JSON de cada idioma para ficheiros mais pequenos.	
Especificação de Saídas a. A divisão não é feita a meio de nenhuma informação de página, portanto não há informações da mesma página em ficheiros diferentes. b. São criados ficheiros com as informações exportadas do site de forma organizada.	
Outros Contorna a limitação de importação de ficheiros grandes no <i>Azure</i> .	
Dependências O ficheiro original JSON da extração tem de estar criado e acessível.	

Tabela 5

Caso de teste 2

ESPECIFICAÇÃO DE CASO DE TESTE 2	
Identificador:	PROJECT-ORG-002
Objetivo:	Garantir que os ficheiros criados cumprem os limites de armazenamento estipulados para a importação no <i>Microsoft Azure</i> .
Autor(es)	Fábio Monteiro
Especificação de Entradas Verificar o peso em megabytes de cada ficheiro.	
Especificação de Saídas a. Nenhum ficheiro criado excede o limite de páginas ditado no script. b. O tamanho de cada ficheiro <i>json</i> fica dentro da margem permitida para importação.	
Outros NA	
Dependências NA	

Tabela 6

Caso de teste 3

ESPECIFICAÇÃO DE CASO DE TESTE 3	
Identificador:	PROJECT-ORG-003
Objetivo:	Validar a integridade estrutural da informação organizada após a divisão.
Autor(es)	Fábio Monteiro
Especificação de Entradas Validar todos os ficheiros resultantes.	
Especificação de Saídas a. Todos os ficheiros são <i>json</i> como esperado e não têm erros de formatação. b. Os ficheiros têm o limite estipulado de entradas, o que prova que não há perda de dados.	
Outros Assegura que nenhuma informação ficou corrompida durante a divisão de ficheiro.	
Dependências NA	

Identificação da intenção do remetente

Tabela 7

Caso de teste 1

ESPECIFICAÇÃO DE CASO DE TESTE 1	
Identificador:	PROJECT-IIR-001
Objetivo:	Verificar a extração correta das palavras-chave a partir de um chatbot criado.
Autor(es)	Fábio Monteiro
Especificação de Entradas Introduzir na interface de chat uma pergunta.	
Especificação de Saídas a. A IA analisa e identifica a intenção principal. b. Demonstra as palavras-chave que retirou da pergunta.	
Outros O teste realizado no chat valida a lógica da extração que será aplicada aos emails.	
Dependências Serviço do Azure OpenAI devidamente criado e interface Streamlit em execução.	

Tabela 8

Caso de teste 2

ESPECIFICAÇÃO DE CASO DE TESTE 2	
Identificador:	PROJECT-IIR-002
Objetivo:	Verificar se a extração está correta, baseado no contexto da pergunta
Autor(es)	Fábio Monteiro
Especificação de Entradas Pergunta acerca de valores de propina de licenciatura em Engenharia Informática.	
Especificação de Saídas a. O sistema compreende o tema e responde devidamente.	
Outros Valida a instrução crítica dada à IA para ler mensagens anteriores quando a questão seguinte do utilizador é vaga.	
Dependências Serviço do Azure OpenAI devidamente criado e interface Streamlit em execução.	

Tabela 9

Caso de teste 3

ESPECIFICAÇÃO DE CASO DE TESTE 3	
Identificador:	PROJECT-IIR-003
Objetivo:	Verificar a utilização dos dois idiomas principais: português e inglês.
Autor(es)	Fábio Monteiro
Especificação de Entradas Pergunta feita em inglês.	
Especificação de Saídas a. O sistema interpreta que o idioma da mensagem é o inglês e identifica a intenção do utilizador. b. A IA responde acertadamente com o idioma da questão.	
Outros Garante que o sistema consegue responder independentemente do idioma.	
Dependências Serviço do Azure OpenAI devidamente criado e interface Streamlit em execução.	

Pesquisa

Tabela 10

Caso de teste 1

ESPECIFICAÇÃO DE CASO DE TESTE 1	
Identificador:	PROJECT-PSQ-001
Objetivo:	Verificar a recuperação de contexto através de correspondência exata de documentos.
Autor(es)	Fábio Monteiro
Especificação de Entradas O sistema envia para o <i>Azure AI Search</i> as palavras-chave específicas.	
Especificação de Saídas a. O serviço de pesquisa acede ao índice vetorial no <i>Azure</i> . b. O sistema devolve os blocos de texto extraídos do documento pdf exato que corresponde esse regulamento.	
Outros Confirma que os pdfs extraídos pelo script foram bem indexados.	
Dependências Serviço do <i>Azure AI Search</i> a funcionar e ficheiros carregados corretamente no índice.	

Tabela 11

Caso de teste 2

ESPECIFICAÇÃO DE CASO DE TESTE 2	
Identificador:	PROJECT-PSQ-002
Objetivo:	Verificar a limitação estrutural do volume de contexto recuperado
Autor(es)	Fábio Monteiro
Especificação de Entradas O sistema envia para o <i>Azure AI Search</i> uma palavra-chave muito comum e que tem correspondência com dezenas de páginas na base de conhecimento.	
Especificação de Saídas a. São feitas avaliações. b. A API devolve três boas opções conforme o que avaliou.	
Outros Este teste é crucial para garantir que não haja limite que possa ser excedido no modelo OpenAI e garante então na mesma rapidez e eficiência.	
Dependências Serviço do <i>Azure AI Search</i> a funcionar e ficheiros carregados corretamente no índice.	

Tabela 12

Caso de teste 3

ESPECIFICAÇÃO DE CASO DE TESTE 3	
Identificador:	PROJECT-PSQ-003
Objetivo:	Verificar o comportamento da IA perante temas mais alheios ao instituto.
Autor(es)	Fábio Monteiro
Especificação de Entradas O sistema envia para o <i>Azure AI Search</i> palavras-chave não relacionadas com a instituição (por exemplo: fala de um problema pessoal)	
Especificação de Saídas a. O serviço de pesquisa não encontra a lógica da questão. b. Retorna na mesma uma resposta com teor institucional, deixando talvez contacto específico da instituição que possa tratar do problema. Por exemplo, referindo que os estudantes podem pedir apoio através do email saude.mental@ispgaya.pt .	
Outros Mecanismos de segurança essencial.	
Dependências Serviço do <i>Azure AI Search</i> a funcionar e ficheiros carregados corretamente no índice.	

Gerar resposta

Tabela 13

Caso de teste 1

ESPECIFICAÇÃO DE CASO DE TESTE 1	
Identificador:	PROJECT-RSP-001
Objetivo:	Verificar a entrega correta de três sugestões de resposta.
Autor(es)	Fábio Monteiro
Especificação de Entradas Enviar um pedido POST com o texto de um email para uma API.	
Especificação de Saídas a. A API retorna o código 'HTTP 200 OK'. b. Apresenta exatamente três opções de resposta distintas. c. O texto obedece às formatações e tipo de escrita previstas.	
Outros Garante a compatibilidade visual caso o texto seja apenas copiado diretamente para o remetente.	
Dependências Extração do contexto bem sucedida e Azure OpenAI funcional.	

Tabela 14

Caso de teste 2

ESPECIFICAÇÃO DE CASO DE TESTE 2	
Identificador:	PROJECT-RSP-002
Objetivo:	Verificar o tempo de resposta ao email recebido.
Autor(es)	Fábio Monteiro
Especificação de Entradas O utilizador envia um email e é necessário aguardar pela receção de soluções de resposta.	
Especificação de Saídas a. Tempo de entrega de soluções de resposta em menos de um minuto. b. Entrega sempre o número correto e de forma rápida e eficaz.	
Outros Assegura que é uma ferramenta que impacta o funcionamento de uma secretaria.	
Dependências Azure OpenAI a funcionar bem e rapidamente.	

Tabela 15

Caso de teste 3

ESPECIFICAÇÃO DE CASO DE TESTE 3	
Identificador:	PROJECT-RSP-003
Objetivo:	Validar o comportamento da IA e regras da instituição perante falta de dados exatos.
Autor(es)	Fábio Monteiro
Especificação de Entradas Um email que pergunta o valor exato de algo cujo contexto não contém em nenhuma tabela de preço específico.	
Especificação de Saídas a. O modelo não inventa valores falsos. b. Segue sempre a regra do sistema de informar que o valor pode rondar 'X' preço se tiver informação para tal, mas sugere sempre contacto direto através dos dados de email e telefónicos do ISPGAYA.	
Outros Assegura que as respostas dadas não fornecem informações falsas.	
Dependências Programação rigorosa do <i>prompt</i> no <i>script</i> da API.	

Cronograma

Neste capítulo será apresentado o cronograma cumprido ao longo do desenvolvimento do projeto com comparação ao planeamento inicial e com as devidas justificações para os incumprimentos registados.

A proposta inicial do projeto previa a utilização de ferramentas *low-code* da Microsoft (Copilot, Azure e Power Automate). Contudo, a necessidade de alterar a arquitetura do sistema do Copilot, devido a limitações na conta do instituto, para uma solução desenvolvida em linguagem Python obrigou a um reajuste significativo das fases de execução e impactou tempos previstos para cada tarefa.

Tabela 16

Cronograma

Fases do projeto	Tempo previsto	Tempo utilizado	Justificação
Estudo das ferramentas da Microsoft e base de <i>Python</i> .	2 semanas	2 semanas	Cumprido: Fase dedicada ao estudo das funcionalidades utilizadas, conforme o plano original.
Recolha e extração de dados.	2 semanas	4 semanas	Atraso de 2 semanas: O desenvolvimento iniciou-se com o <i>Power Automate</i> para a extração automatizada dos dados do <i>website</i> . Contudo, concluiu-se que não era a ferramenta ideal para realizar essa tarefa e optou-se por alterar para <i>scripts</i> de <i>python</i> .
Integração da IA	3 semanas	5 semanas	Atraso de 2 semanas: Após a extração, a tentativa de integrar os dados no <i>Microsoft Copilot</i> foi uma má decisão devido a certas limitações recentes da <i>Microsoft</i> e da conta institucional. Foi necessário aguardar por confirmação do lado do instituto, com a esperança de desbloquear a ferramenta, mas foi mesmo necessário alterar todo o sistema e desenvolver uma <i>API</i> de emails em <i>Python</i> e serviços <i>cloud</i> do <i>Azure</i> .
Testes e Validação	2 semanas	2 semanas	Cumprido: A validação de sugestões de resposta a emails e validação manteve-se dentro do tempo estipulado.
Desenvolvimento do relatório	2 semanas	3 semanas	Atraso de 1 semana: As sucessivas alterações exigiram um esforço extra na documentação, com a necessidade de reestruturação nos diagramas <i>UML</i> e justificação das decisões tomadas no documento final.

Problemas e decisões

Ao longo deste projeto foram surgindo problemas e foram necessárias algumas tomadas de decisão para garantir o cumprimento de todos os objetivos propostos.

Limitações na extração de dados com o *Power Automate*

- Problema detetado: A fase inicial previa a utilização do *Power Automate* para extrair e organizar os dados do website do ISPGAYA. Durante os primeiros testes práticos, notou-se que esta ferramenta apresentava excessiva carga de trabalho e falta de recursos avançados para lidar com o webscraping.
- Decisão tomada: Abandonar a utilização da ferramenta para desenvolver um ficheiro python para extração automática dos dados.
- Justificação: Com a utilização das bibliotecas especializadas, garantiu-se uma melhor extração de dados e mais rápida.

Restrições no Microsoft Copilot

- Problema detetado: O objetivo central do projeto consistia em integrar o sistema no *Microsoft 365* e utilizar o *Microsoft Copilot* para gerar sugestões de resposta automáticas aos emails recebidos. Contudo, a conta institucional de aluno do ISPGAYA possui limitações, o que impossibilita a utilização da ferramenta prevista.
- Decisão tomada: Substituir o *Microsoft Copilot* com o desenvolvimento de um ficheiro *python* com uma *API* com integração direta aos serviços *cloud* do *Azure*.
- Justificação: Esta foi a decisão mais crítica do projeto, pois foi a única ferramenta não utilizada que constava no plano do projeto. O *Azure AI Search* permitiu vetorizar e indexar a documentação extraída e o *Azure OpenAI* faz a interpretação dos emails recebidos e gera soluções de resposta. Portanto, a criação da *API* desta forma garante na mesma que o sistema funcione como previsto, independentemente de não estar a utilizar o *Microsoft Copilot*.

Limitações de espaço de ficheiros importados no Azure

- Problema detetado: Após a extração bem sucedida de todos os dados do site do instituto, o ficheiro *JSON* resultante era demasiado grande para importar no *Microsoft Azure* de uma só vez.
- Decisão tomada: Desenvolver um script para ser feita a fragmentação dos dados exportados do ficheiro *json* em mais do que um ficheiro, ao ponto de conseguir importar toda a informação.
- Justificação: A divisão em ficheiros menores foi essencial para a continuação do projeto por respeitar os limites do *Microsoft Azure*.

Análise de resultados

Neste capítulo será efetuada uma análise detalhada dos resultados obtidos com a concretização deste projeto.

Os resultados demonstram que, apenas de certas alterações significativas, o propósito central do projeto foi alcançado com sucesso.

1. Criação e Estruturação da Base de Conhecimento

Um dos objetivos iniciais consistia na recolha da informação disponível no website institucional (www.ispgaya.pt) para a criação de uma base de conhecimento. Este objetivo foi cumprido.

Os scripts desenvolvidos conseguiram percorrer autonomamente o site e no repositório de documentos no site, o sistema acedeu com sucesso às pastas e transferiu os ficheiros *pdfs*.

As informações do site foram estruturadas e guardadas em ficheiros *json*. Para garantir que funcionava no *Azure*, os dados foram segmentados em ficheiros, o que assegurou ter um peso ideal para importar sem falhas.

2. Gerar respostas

A proposta previa o desenvolvimento de mecanismos de interpretação de emails para identificar intenções e gerar soluções de respostas contextualizadas. O trabalho desenvolvido validou plenamente esse requisito.

O sistema provou ser altamente eficaz na extração das palavras-chave do email e, ao receber a mensagem, a *API* pesquisa no índice do *Azure* e pega nos textos e documentos relevantes para formar uma resposta de contexto institucional.

Pada cada email processado, o modelo de linguagem gera três possíveis respostas ao email recebido. Mantém o texto formatado e respeita as diretrizes rigorosas do *prompt* dado, que faz com que inclua sempre os contactos oficiais do instituto e adapta ao idioma da conversa, por exemplo.

3. Impacto na Eficácia da Comunicação

O objetivo principal e final do projeto era melhorar a eficácia da comunicação entre alunos e secretaria, tanto como em reduzir o tempo de resposta do atendimento via email. A implementação deste projeto mostra um impacto bastante positivo.

Um funcionário da secretaria, ao receber um email sobre qualquer tema, pode necessitar de tempo para pesquisar o assunto em documentos de regulamentos do instituto, confirmar valores e enviar a resposta formalmente. Este processo pode demorar, em média, entre cinco e dez minutos.

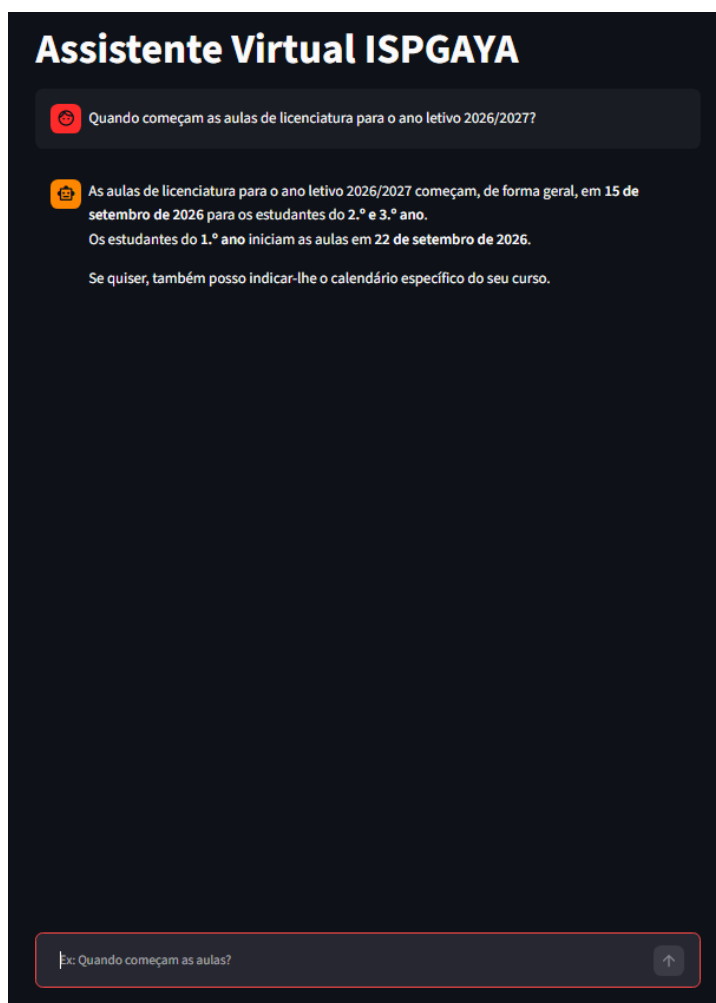
Desde o tempo de receção do email do estudante até receber as sugestões de resposta demora menos de um minuto, são apenas uns segundos. Isto demonstra o quão eficaz é e que ajuda muito no dia a dia.

Demonstração visual do projeto

Muitas das validações do projeto foi feita através do chat criado com a IA desenvolvida.

Figura 4

Imagem do chatbot criado com exemplo de pergunta e resposta.



Depois de todos os testes, foi feito o desenvolvimento para funcionar com troca de emails.

Como mostra as duas figuras apresentadas abaixo, quando existe a receção de um email, rapidamente o sistema funciona de forma a enviar resposta de soluções em menos de um minuto.

Figura 5

Receção dos emails, do utilizador e da resposta da IA, em menos de um minuto.

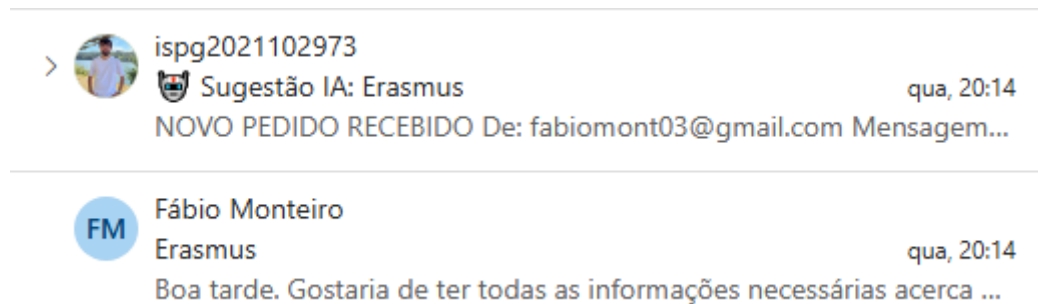
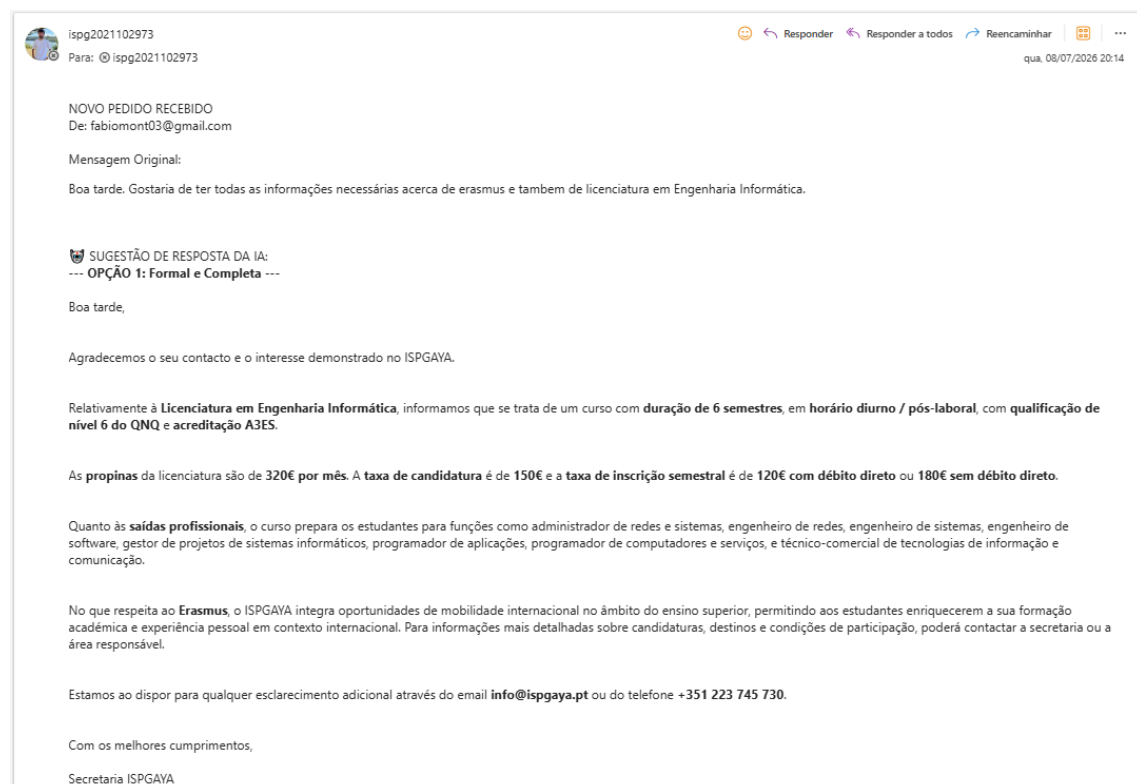


Figura 6

Exemplo de email recebido de apoio de respostas.



Conclusões

O presente projeto teve como objetivo o desenvolvimento de um sistema inteligente capaz de apoiar a gestão e automatizar a comunicação institucional do ISPGAYA por e-mail. Através do trabalho desenvolvido, procurou-se transformar os dados do website e dos regulamentos do instituto numa ferramenta capaz de gerar sugestões de resposta precisas e contextualizadas.

Numa perspetiva global dos objetivos inicialmente propostos, constata-se que o propósito central do projeto foi alcançado com êxito. O sistema é capaz de recolher a informação, interpretar os e-mails recebidos, identificar as intenções do remetente e dar três opções de resposta coerentes, formatadas e no idioma correto. O único objetivo que não foi cumprido através do planeamento original foi a integração direta no Microsoft Copilot. Esta alteração foi necessária devido às limitações surgidas. Numa primeira fase, a extração de dados através do Power Automate foi também limitada face à complexidade das páginas e dos documentos PDF. Estes problemas trouxeram à necessidade de fazer mudanças no projeto, foi optado então por fazer o desenvolvimento de uma solução em linguagem de programação Python, que é suportada pelos serviços cloud do Azure (Azure AI Search e Azure OpenAI).

O ponto principal deste projeto é a demonstração prática de que é possível modernizar drasticamente o atendimento da secretaria e reduzir o tempo de resposta a emails, sem comprometer a fiabilidade da informação prestada. Mesmo assim, para garantir confiabilidade, ainda requer execução manual. Pode, ainda assim, perspetivar-se como evolução futura a automatização destes emails, após meses de utilização desta ferramenta.

Em reflexão crítica, o período de desenvolvimento deste projeto foi uma fase de enorme crescimento técnico e pessoal. A principal dificuldade enfrentada foi gerir a incerteza e a frustração quando as ferramentas planeadas falharam estavam bloqueadas. No entanto, essas mesmas dificuldades converteram-se nas aprendizagens mais significantes do percurso. A necessidade de abandonar ferramentas *low-code* e desenvolver a solução de raiz obrigou a uma aprendizagem de conhecimentos em programação *Python*, técnicas avançadas de *Web Scraping*, *Prompts* e criação de *APIs* na *cloud*. Resumidamente, o projeto comprovou que a adaptabilidade e a resiliência são competências indispensáveis na engenharia informática.

Bibliografia

Mitchell, R. (2019). *Web Scraping com Python–2ª edição: Coletando mais dados da web moderna*. Novatec Editora.

Borges, L. E. (2014). *Python para desenvolvedores: aborda Python 3.3*. Novatec Editora.

Larguesa, R. P. (2025). *Engenharia de prompt para DEVS: um guia para aprender a usar a IA antes que a IA aprenda a usar você*. Casa do Código.

Alves, F. R., & Alcalá, S. G. S. (2022). Análise da abordagem LOW-CODE como facilitador da transformação digital em indústrias. *Revista e-TECH: Tecnologias para Competitividade Industrial-ISSN-1983-1838*, 15(1).

ISPGAYA. (2022, 29 de julho). *4th International Congress on Psychology, Education and Culture* [fotografia].

Glossário

Web Scraping: Técnica de extração automatizada de dados de websites.

Script: Ficheiro de texto que contém um conjunto de instruções de código.

Prompt: Instrução de texto, comando ou conjunto de regras fornecido a um modelo de Inteligência Artificial para contextualizar e guiar na execução de uma tarefa específica.

Low-code: Desenvolvimento de software que requer pouca ou nenhuma linha de código realizada manualmente, utiliza apenas interfaces visuais e componentes de fácil configuração para criar aplicações de forma mais rápida.

Apêndices/anexos

Ilustração 1

Fotografia do exterior do ISPGAYA, retirada da página web do instituto que fala acerca da ‘4th International Congress on Psychology, Education and Culture’

